

FACULDADE ESUDA

ESUDA CERTIFICADOS

Disciplina: Desenvolvimento Backend

Turma: ADS Manhã

Integrantes: Thiago Henrique Ciriano Neves

Professor(a): Victor Oliveira

SUMÁRIO

1. APRESENTAÇÃO E CONTEXTO DO PROBLEMA	1
1.1 Qual problema o sistema resolve	1
1.2 Quem vai usar	2
1.3 Objetivo da API	2
1.4 Escopo da primeira versão	3
2. REQUISITOS DO SISTEMA	4
3. PERFIS DE USUÁRIO	4
3.1 Por que definir isso desde já?	5
4. REGRAS DE NEGÓCIO	5
5. MODELO DE DADOS (DER)	6
5.1 Relacionamentos e cardinalidades	7
5.2 Decisões de modelagem	8
6. DICIONÁRIO DE DADOS	8
6.1 Tabela usuarios	9
6.2 Tabela categorias	9
6.3 Tabela eventos	9
6.4 Tabela inscricoes	10
6.5 Tabela certificados	10
7. ROTAS DA API, CONTRATOS E STATUS CODES	10
7.1 Lista de rotas	11
7.2 Contratos de entrada e saída	11
POST /api/v1/usuarios	11
POST /api/v1/eventos	13
POST /api/v1/inscricoes	16
PUT /api/v1/inscricoes/{inscricao_id}/cancelar	18
POST /api/v1/certificados	19
POST /api/v1/categorias	21

7.3 Status codes usados	22
7.4 Padrão de resposta e de erro	23
8. MATRIZ DE PERMISSÕES	23
9. ESTRUTURA DO PROJETO E TECNOLOGIAS	24
9.1 Estrutura de pastas	25
9.2 Tecnologias e justificativa	28
10. ESTRATÉGIA DE BANCO DE DADOS E MIGRATIONS	29
11. BACKLOG DO PROJETO	30
12. CRITÉRIOS DE ACEITAÇÃO	30
12.1 Cadastro de usuário (US01)	31
12.2 Criação de evento (US04)	32
12.3 Inscrição em evento (US07)	33
12.4 Cancelamento de inscrição (US08)	34
12.5 Emissão de certificado (US10)	34
12.6 Exclusões	35
12.7 Documentação (RF16)	36
13. COMO RODAR O PROJETO	36
14. CONSIDERAÇÕES FINAIS	37

Obs: este sumário é gerado por códigos de campo. Ao abrir o documento, clique com o botão direito sobre o sumário e escolha "Atualizar campo" para corrigir os números de página.

1. APRESENTAÇÃO E CONTEXTO DO PROBLEMA

Este documento reúne a documentação da primeira entrega do projeto Esuda Certificados, uma API REST feita em Python com FastAPI para gerenciar eventos acadêmicos, inscrições e emissão de certificados. Aqui estão descritos o problema que motivou o sistema, os requisitos, os perfis de usuário, as regras de negócio, o modelo de dados com o DER, o dicionário de dados, os contratos das rotas, a matriz de permissões, a estrutura do projeto, o backlog e os critérios de aceitação.

1.1 Qual problema o sistema resolve

Hoje a organização dos eventos acadêmicos da instituição é feita de um jeito bem manual. As inscrições dos alunos ficam em planilhas e listas no papel, os organizadores controlam as vagas "de cabeça" e, quando o evento acaba, os certificados são emitidos um por um no editor de texto. Isso acaba gerando problemas recorrentes, que listamos abaixo a partir do que observamos no dia a dia da instituição:

- aluno que se inscreve duas vezes no mesmo evento, porque ninguém controla a lista;
- eventos que passam da capacidade, porque a vaga não é conferida na hora;
- certificado que demora ou é emitido para quem nem compareceu nem estava inscrito;
- nenhuma história do que aconteceu, porque cada organizador guarda os dados do seu jeito.

A ideia do Esuda Certificados é centralizar tudo isso em uma API: os eventos ficam cadastrados num lugar só, a inscrição respeita as regras (vaga, duplicidade, data) e o certificado é emitido direto da inscrição, com um código único. Assim o controle deixa de depender de conferência manual e passa a ser feito pelo próprio sistema.

1.2 Quem vai usar

São três tipos de usuário, que descrevemos com mais detalhes no capítulo 3. Resumindo: o **Admin** é a coordenação ou a secretaria, que gerencia usuários, eventos e pode emitir certificados; o **Organizador** é o professor ou responsável por

um evento, que cria e acompanha os inscritos; e o **Participante** é o aluno, que consulta os eventos e faz a própria inscrição.

1.3 Objetivo da API

O objetivo é oferecer uma API REST que cubra o fluxo completo do processo: cadastro de usuário, cadastro de evento, inscrição e emissão de certificado, aplicando as regras de negócio no meio do caminho para não depender de conferência manual. Cada etapa desse fluxo tem rotas próprias, documentadas no capítulo 7, e as regras que protegem cada operação estão reunidas no capítulo 4.

1.4 Escopo da primeira versão

Nesta primeira entrega o foco é a base do sistema, sem login. O que já funciona:

- cadastro, consulta, alteração e exclusão de usuários, categorias e eventos;
- inscrição em evento e cancelamento de inscrição;
- emissão e consulta de certificados com código único;
- todas as regras de negócio já validadas nas rotas;
- documentação automática pelo Swagger.

O que fica para a segunda unidade: login com JWT, controle de acesso por perfil (a matriz de permissões já está pronta no capítulo 8), criptografia forte da senha e os testes automatizados com Pytest. Deixamos tudo isso escrito no backlog do capítulo 11 para mostrar que já estava planejado desde o começo.

2. REQUISITOS DO SISTEMA

Levantamos os requisitos pensando no fluxo que descrevemos no capítulo 1 (evento, inscrição e certificado). Separamos em requisitos funcionais, que dizem o que o sistema faz, e requisitos não funcionais, que dizem como o sistema precisa trabalhar. Cada requisito tem um identificador para facilitar a citação nos outros capítulos.

Tabela 1 – Requisitos funcionais

ID	Requisito
----	-----------

RF01	A API deve permitir cadastrar usuário informando nome, e-mail e senha.
RF02	A API não deve permitir dois usuários com o mesmo e-mail.
RF03	A API deve permitir listar, consultar por id, atualizar e excluir usuários.
RF04	A API deve permitir cadastrar, listar, atualizar e excluir categorias de evento.
RF05	A API deve permitir cadastrar evento informando título, descrição, data, local, capacidade e categoria (opcional).
RF06	A API deve permitir listar os eventos (com filtro por status) e consultar um evento pelo id.
RF07	A API deve permitir atualizar os dados de um evento e cancelá-lo (mudança de status).
RF08	A API deve permitir excluir um evento, desde que ele não tenha inscrições.
RF09	A API deve permitir inscrever um usuário em um evento, validando as regras de negócio (RN02 a RN05 e RN09).
RF10	A API deve permitir listar os inscritos ativos de um evento.
RF11	A API deve permitir listar todas as inscrições de um usuário.
RF12	A API deve permitir cancelar uma inscrição (mudando o status, sem apagar o registro).
RF13	A API deve permitir emitir certificado para uma inscrição ativa, gerando um código único.
RF14	A API deve permitir listar e consultar os certificados emitidos.
RF15	A API deve validar os dados de entrada e devolver mensagens de erro claras.
RF16	A API deve ter documentação automática acessível pelo Swagger (/docs).

Tabela 2 – Requisitos não funcionais

ID	Requisito
RNF01	Os dados de entrada devem ser validados automaticamente com Pydantic (tipo, campo obrigatório, e-mail válido, capacidade positiva).
RNF02	A API deve seguir o padrão REST e ter versionamento na URL (/api/v1/...).
RNF03	A documentação do Swagger deve estar sempre atualizada, gerada direto do código.
RNF04	A senha nunca pode ser devolvida em nenhuma resposta da API.

RNF05	As mudanças de banco devem ser versionadas com migrations (Alembic).
RNF06	O código deve ser organizado em camadas separadas (rotas, schemas, models, configuração e banco), para facilitar a manutenção.
RNF07	Os erros devem ter um padrão único de resposta (<code>{"detail": "mensagem"}</code>).
RNF08	O projeto deve rodar em qualquer máquina com Python 3.10+ sem depender de serviços externos (banco SQLite por padrão, configurável para PostgreSQL).

3. PERFIS DE USUÁRIO

Mesmo sabendo que a autorização com login só vai ser implementada na segunda unidade, já definimos os três perfis agora, porque eles influenciam várias decisões da primeira entrega: o campo papel na tabela de usuários, a matriz de permissões do capítulo 8 e a validação de algumas regras. A tabela abaixo resume cada perfil e o que ele faz no sistema.

Tabela 3 – Perfis de usuário do sistema

Perfil	Descrição	O que faz no sistema
Admin	Coordenação / secretaria da instituição	Gerencia usuários, categorias e eventos, e pode emitir certificados e excluir eventos
Organizador	Professor ou responsável por um evento	Cria e organiza os próprios eventos, acompanha a lista de inscritos e emite certificados
Participante	Aluno da instituição	Consulta os eventos, faz a própria inscrição e cancela a própria inscrição

3.1 Por que definir isso desde já?

Porque se deixássemos para pensar em perfis só na hora de implementar o JWT, teríamos que mexer no banco, no cadastro de usuário e em várias regras de uma vez. Dessa forma o campo papel já nasce na tabela de usuários (com valor padrão participante) e a matriz de permissões do capítulo 8 já serve de guia para a segunda entrega. Foi uma decisão que tomamos depois de conversar em sala sobre a ordem das entregas.

Uma observação importante: nesta primeira versão a API ainda não bloqueia nada por perfil, porque não existe login ainda. Isso está de acordo com o que foi combinado na disciplina, e o README do projeto avisa que a autorização vem na unidade 2. Ou seja, hoje qualquer pessoa consegue chamar as rotas, e a checagem de permissão será acrescentada por cima sem precisar refazer nada.

4. REGRAS DE NEGÓCIO

Aqui estão as regras que o sistema precisa obedecer. Marcamos também onde cada uma é validada hoje, porque achamos que isso ajuda na hora da defesa do projeto. Todas as regras que têm retorno de código de status já foram testadas de ponta a ponta pelo Swagger.

Tabela 4 – Regras de negócio (RN01 a RN15)

ID	Regra	Onde é validada
RN01	O e-mail do usuário deve ser único no sistema.	Rota de criar usuário (e índice unique no banco)
RN02	O formato do e-mail deve ser válido.	Validação do Pydantic (EmailStr), retorna 422
RN03	Um usuário não pode se inscrever duas vezes no mesmo evento (inscrição ativa).	Rota de criar inscrição, retorna 400
RN04	Um evento não pode ultrapassar a capacidade máxima de inscritos ativos.	Rota de criar inscrição, conta as vagas usadas, retorna 400
RN05	A capacidade do evento deve ser maior que zero.	Validação no schema, retorna 422
RN06	Não é permitido cadastrar evento com data passada.	Validação no schema, retorna 422
RN07	Todo evento precisa de título, data, local e capacidade.	Campos obrigatórios no schema, retorna 422
RN08	Só pode se inscrever em evento com status ativo.	Rota de criar inscrição, retorna 400
RN09	Não pode se inscrever em evento cuja data já passou.	Rota de criar inscrição, retorna 400

RN10	Um participante só pode cancelar a própria inscrição (a conferência de "ser o dono" será feita com JWT na unidade 2).	Cancelamento muda o status para cancelada, nunca apaga o registro
RN11	Um certificado só pode ser emitido para inscrição ativa.	Rota de emitir certificado, retorna 400
RN12	Não pode existir mais de um certificado para a mesma inscrição.	Rota de emitir certificado (e unique no banco), retorna 400
RN13	A senha do usuário nunca pode ser devolvida em resposta nenhuma.	O schema de resposta não tem o campo senha
RN14	Um evento só pode ser excluído se não tiver inscrições cadastradas (para não perder o histórico).	Rota de excluir evento, retorna 400
RN15	Apenas administradores podem excluir eventos.	Será aplicado na unidade 2 com JWT; a matriz de permissões já define isso

Uma observação sobre o processo: no começo o projeto tinha poucas regras e era basicamente um CRUD simples. Quando listamos essas regras com calma, o sistema ficou bem mais parecido com algo de verdade, porque é exatamente aqui que a API "pensa" antes de salvar. Foi por isso que decidimos dedicar um capítulo só para elas, em vez de espalhar as regras pelo texto.

5. MODELO DE DADOS (DER)

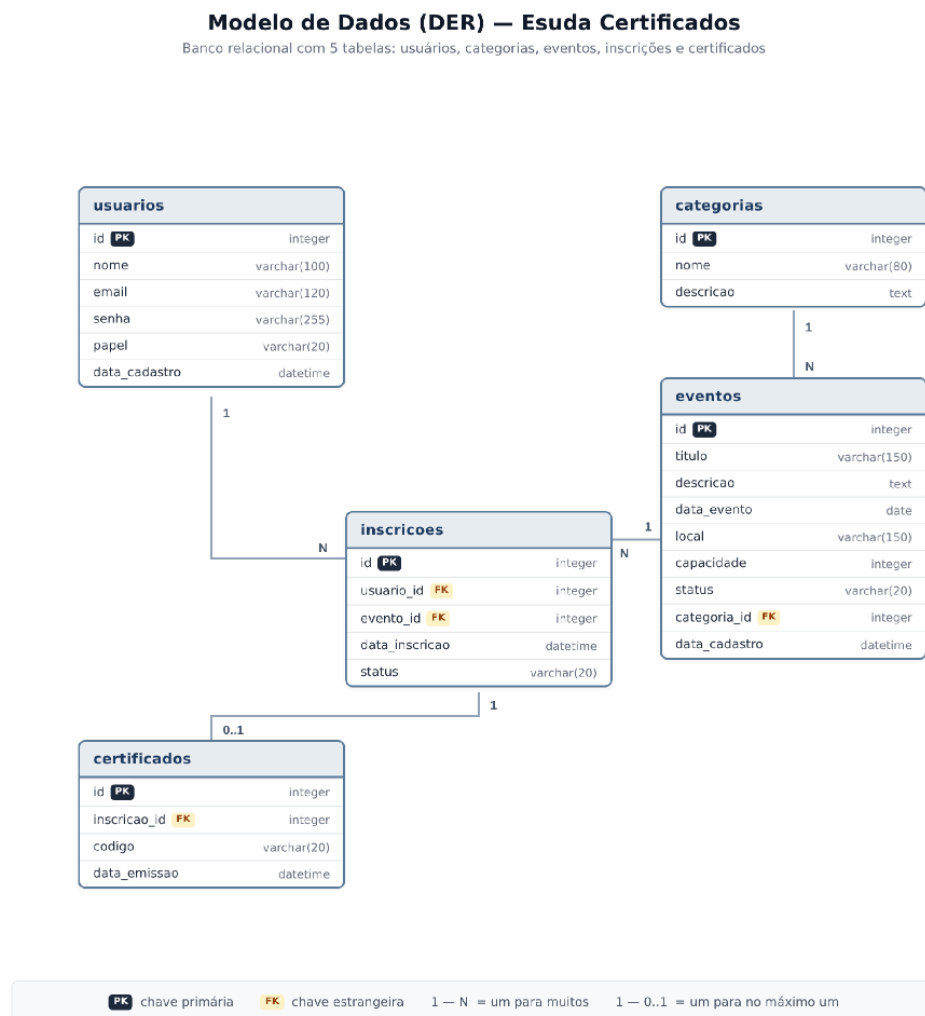
Escolhemos 5 entidades pensando direto no fluxo do sistema. Cada uma tem um papel claro:

- **usuarios:** qualquer pessoa que usa a API (aluno, organizador ou admin). O campo papel guarda o perfil;
- **categorias:** tipo do evento (workshop, palestra, semana acadêmica...). Serve só para organizar e filtrar;
- **eventos:** o coração do sistema, guarda data, local, capacidade e status;

- **inscricoes:** liga um usuário a um evento. É uma tabela de relacionamento, com data e status próprios (ativa/cancelada);
- **certificados:** nasce a partir de uma inscrição e guarda o código único de validação.

Pensamos em colocar também uma entidade de "Atividade" (palestras dentro de um evento), mas decidimos deixar para uma versão futura porque o escopo da primeira entrega já estava razoável e não queremos entregar coisa pela metade. O diagrama abaixo mostra as cinco tabelas, suas chaves e os relacionamentos.

Figura 1 – Diagrama Entidade-Relacionamento (DER) do sistema



5.1 Relacionamentos e cardinalidades

A tabela abaixo resume os quatro relacionamentos do modelo e por que existem. Repare que o relacionamento entre inscrições e certificados é 1 para 0..1, porque uma inscrição pode chegar a ter no máximo um certificado, mas pode nunca ter.

Tabela 5 - Relacionamentos e cardinalidades

Relacionamento	Cardinalidade	Explicação
categorias → eventos	1 : N	Uma categoria pode ter vários eventos. Um evento tem no máximo uma categoria, e pode ficar sem categoria nenhuma (campo opcional)
usuários → inscrições	1 : N	Um usuário pode ter várias inscrições (uma em cada evento)
eventos → inscrições	1 : N	Um evento pode receber várias inscrições, limitadas pela capacidade
inscrições → certificados	1 : 0..1	Uma inscrição pode ter no máximo um certificado, e só existe certificado ligado a uma inscrição

5.2 Decisões de modelagem

Também vale explicar duas decisões de modelagem que tomamos durante o desenho do banco, porque elas aparecem nas regras do capítulo 4:

1. **Cancelamento não apaga inscrição:** quando o participante cancela, o campo status muda para cancelada e o registro continua no banco. Assim mantemos o histórico (RN10);
2. **Certificado ligado à inscrição, não ao usuário:** emitimos o certificado a partir da inscrição, porque é a inscrição que garante que a pessoa participava daquele evento (RN11 e RN12).

6. DICIONÁRIO DE DADOS

Este capítulo detalha campo por campo as 5 tabelas do banco. A coluna "obrigatório" considera o que o banco e o Pydantic exigem na hora de receber os dados, e a coluna de observações aponta as regras de negócio ligadas a cada campo.

6.1 Tabela usuarios

Tabela 6 – Dicionário de dados da tabela usuarios

Campo	Tipo	Obrigatório	Regra / Observação
id	integer	Sim (auto)	Chave primária
nome	varchar(100)	Sim	Nome completo do usuário
email	varchar(120)	Sim	Único no sistema e com formato válido (RN01 e RN02)
senha	varchar(255)	Sim	Guardada como hash; nunca volta em resposta (RN13)
papel	varchar(20)	Sim	admin, organizador ou participante. Padrão: participante
data_cadastro	datetime	Sim (auto)	Preenchido na criação

6.2 Tabela categorias

Tabela 7 – Dicionário de dados da tabela categorias

Campo	Tipo	Obrigatório	Regra / Observação
id	integer	Sim (auto)	Chave primária
nome	varchar(80)	Sim	Único (não pode repetir categoria)
descricao	text	Não	Descrição livre

6.3 Tabela eventos

Tabela 8 – Dicionário de dados da tabela eventos

Campo	Tipo	Obrigatório	Regra / Observação
id	integer	Sim (auto)	Chave primária
titulo	varchar(150)	Sim	Nome do evento
descricao	text	Não	Detalhes do evento

data_vento	date	Sim	Não pode ser data passada (RN06)
local	varchar(150)	Sim	Sala, laboratório, auditório...
capacidade	integer	Sim	Maior que zero (RN05)
status	varchar(20)	Sim	ativo ou cancelado. Padrão: ativo
categoria_id	integer	Não	Chave estrangeira para categorias.id
data_cadastrado	datetime	Sim (auto)	Preenchido na criação

6.4 Tabela inscricoes

Tabela 9 – Dicionário de dados da tabela inscricoes

Campo	Tipo	Obrigatório	Regra / Observação
id	integer	Sim (auto)	Chave primária
usuario_id	integer	Sim	Chave estrangeira para usuarios.id
evento_id	integer	Sim	Chave estrangeira para eventos.id
data_inscricao	datetime	Sim (auto)	Momento da inscrição
status	varchar(20)	Sim	ativa ou cancelada. Padrão: ativa. Cancelar não apaga (RN10)

6.5 Tabela certificados

Tabela 10 – Dicionário de dados da tabela certificados

Campo	Tipo	Obrigatório	Regra / Observação
id	integer	Sim (auto)	Chave primária
inscricao_id	integer	Sim	Chave estrangeira para inscricoes.id e única (RN12)

codigo	varchar(20)	Sim	Código único gerado na emissão, usado para validar o certificado
data_emissao	datetime	Sim (auto)	Momento da emissão

7. ROTAS DA API, CONTRATOS E STATUS CODES

Todas as rotas começam com /api/v1 (versionamento na URL), como mostra o requisito RNF02. A lista abaixo é a mesma que aparece no Swagger quando o servidor está rodando, agrupada por entidade. Ao todo são 22 rotas entre as cinco entidades.

7.1 Lista de rotas

Tabela 11 - Lista de rotas da API

Método	Rota	O que faz	Status
POST	/api/v1/usuarios	Cadastra usuário	201
GET	/api/v1/usuarios	Lista usuários (com skip/limit)	200
GET	/api/v1/usuarios/{usuario_id}	Consulta um usuário	200
PUT	/api/v1/usuarios/{usuario_id}	Atualiza um usuário	200
DELETE	/api/v1/usuarios/{usuario_id}	Exclui um usuário	204
POST	/api/v1/eventos	Cadastra evento	201
GET	/api/v1/eventos	Lista eventos (filtro opcional por status)	200
GET	/api/v1/eventos/{evento_id}	Consulta um evento	200
PUT	/api/v1/eventos/{evento_id}	Atualiza um evento	200
DELETE	/api/v1/eventos/{evento_id}	Exclui um evento	204
POST	/api/v1/inscricoes	Inscribe usuário em evento	201
GET	/api/v1/inscricoes	Lista todas as inscrições	200
GET	/api/v1/inscricoes/evento/{evento_id}	Lista os inscritos ativos de um evento	200

GET	/api/v1/inscricoes/usuario/{usuario_id}	Lista as inscrições de um usuário	200
PUT	/api/v1/inscricoes/{inscricao_id}/cancelar	Cancela uma inscrição	200
POST	/api/v1/certificados	Emite certificado	201
GET	/api/v1/certificados	Lista certificados	200
GET	/api/v1/certificados/{certificado_id}	Consulta um certificado	200
POST	/api/v1/categorias	Cadastra categoria	201
GET	/api/v1/categorias	Lista categorias	200
PUT	/api/v1/categorias/{categoria_id}	Atualiza uma categoria	200
DELETE	/api/v1/categorias/{categoria_id}	Exclui uma categoria	204

7.2 Contratos de entrada e saída

Nesta seção estão os contratos das rotas principais, no formato JSON, exatamente como o Swagger mostra. Os contratos das outras rotas (GET, PUT e DELETE que não aparecem aqui) seguem o mesmo formato dos objetos apresentados abaixo.

POST /api/v1/usuarios

Entrada:

```
{
  "nome": "João Silva",
  "email": "joao@email.com",
  "senha": "123456",
  "papel": "participante"
}
```

Saída (201) – reparar que a senha nunca volta (RN13):

```
{
  "id": 1,
  "nome": "João Silva",
  "email": "joao@email.com",
  "papel": "participante",
  "data_cadastro": "2026-09-13T10:32:11"
}
```

O GET /api/v1/usuarios devolve status 200 com uma lista de objetos iguais ao do exemplo acima. Já o PUT /api/v1/usuarios/{usuario_id} recebe só os campos que quiser

alterar (por exemplo {"nome": "João da Silva"}) e devolve o usuário já atualizado com status 200.

POST /api/v1/eventos

Entrada:

```
{
  "titulo": "Workshop de FastAPI",
  "descricao": "Evento prático sobre APIs",
  "data_evento": "2026-11-10",
  "local": "Laboratório 3",
  "capacidade": 50,
  "categoria_id": 1
}
```

Saída (201):

```
{
  "id": 1,
  "titulo": "Workshop de FastAPI",
  "descricao": "Evento prático sobre APIs",
  "data_evento": "2026-11-10",
  "local": "Laboratório 3",
  "capacidade": 50,
  "status": "ativo",
  "categoria_id": 1,
  "data_cadastro": "2026-09-13T10:35:00"
}
```

O PUT /api/v1/eventos/{evento_id} aceita campos opcionais e é por ele que o evento é cancelado, mudando o status para cancelado (por exemplo {"status": "cancelado"}). A resposta é o evento atualizado com status 200.

POST /api/v1/inscricoes

Entrada:

```
{
  "usuario_id": 1,
  "evento_id": 1
}
```

Saída (201):

```
{
  "id": 1,
  "usuario_id": 1,
  "evento_id": 1,
  "data_inscricao": "2026-09-13T10:40:00",
  "status": "ativa"
}
```

PUT /api/v1/inscricoes/{inscricao_id}/cancelar

Não recebe corpo. É a rota de cancelamento, que muda o status sem apagar o registro (RN10). Saída (200):

```
{
  "id": 1,
  "usuario_id": 1,
  "evento_id": 1,
  "data_inscricao": "2026-09-13T10:40:00",
  "status": "cancelada"
}
```

POST /api/v1/certificados

Entrada:

```
{
  "inscricao_id": 2
}
```

Saída (201) – o código é gerado pelo sistema e serve para validar o certificado:

```
{
  "id": 1,
  "inscricao_id": 2,
  "codigo": "A3F19C2B",
  "data_emissao": "2026-09-13T10:45:00"
}
```

POST /api/v1/categorias

Entrada:

```
{
  "nome": "Workshop",
  "descricao": "Eventos práticos"
}
```

Saída (201):

```
{
  "id": 1,
  "nome": "Workshop",
  "descricao": "Eventos práticos"
}
```

Todas as listagens (usuarios, eventos, inscricoes, certificados, categorias e as variações de inscrição) devolvem um array JSON com os mesmos objetos mostrados acima, com status 200. As rotas DELETE não têm corpo de resposta e devolvem status 204.

7.3 Status codes usados

A tabela abaixo reúne os códigos de status HTTP que a API devolve e quando cada um acontece. Os códigos 401 e 403 vão começar a aparecer a partir da unidade 2, quando entrar o login com JWT.

Tabela 12 - Códigos de status usados pela API

Situação	Status
Registro criado	201
Consulta/atualização realizada	200
Exclusão realizada	204
Erro de regra de negócio (ex.: evento lotado)	400
Registro não encontrado	404
Erro de validação dos campos	422
Não autenticado (a partir da unidade 2, com JWT)	401
Sem permissão (a partir da unidade 2, com JWT)	403

7.4 Padrão de resposta e de erro

Nosso padrão é o seguinte:

- **Sucesso:** a rota devolve direto o objeto (ou a lista de objetos) criado/consultado;
- **Erro:** devolve sempre {"detail": "mensagem do erro"}.

Justificativa da escolha: testamos a possibilidade de colocar tudo dentro de um envelope tipo {"success": true, "data": {...}}, mas decidimos seguir o padrão que o próprio FastAPI já traz. Os motivos: o erro {"detail": ...} já é o comportamento nativo do HTTPException, o Swagger mostra exatamente o que a rota devolve (sem camada extra de tradução na cabeça de quem vai consumir) e o código fica mais enxuto. Acreditamos que um padrão simples, documentado e consistente vale mais do que um

envelope bonito mas cheio de campo redundante, já que o status HTTP já diz se deu certo ou não.

8. MATRIZ DE PERMISSÕES

A autorização de verdade (quem pode chamar o quê) só entra na segunda unidade, junto com o login JWT. Mesmo assim já deixamos a matriz pronta, porque ela orientou o desenho das rotas e está ligada direto às regras RN10 e RN15. Na tabela, cada linha é uma funcionalidade e as colunas mostram quem pode executá-la.

Tabela 13 – Matriz de permissões por perfil

Funcionalidade	Admin	Organizador	Participante
Se cadastrar no sistema	Sim	Sim	Sim
Gerenciar usuários (alterar/excluir outros)	Sim	Não	Não
Criar evento	Sim	Sim	Não
Editar evento	Sim	Sim (só os próprios, com JWT)	Não
Excluir evento	Sim	Não	Não
Listar/consultar eventos	Sim	Sim	Sim
Inscriver-se em evento	Não	Não	Sim
Cancelar a própria inscrição	Sim	Sim	Sim
Ver lista de inscritos de um evento	Sim	Sim	Não
Emitir certificado	Sim	Sim	Não
Gerenciar categorias	Sim	Sim	Não

Observações que achamos importante deixar escritas:

- Inscrever-se é uma ação do participante. Admin e organizador não se inscrevem em evento pelo sistema (isso segue a matriz exemplo passada em aula);
- Excluir evento é coisa de admin (RN15). Organizador que quiser "tirar um evento do ar" usa o cancelamento por status, que mantém o histórico;
- Na primeira entrega essas regras ainda não são bloqueadas no código, porque não existe login. O campo papel já está salvo no banco, então a checagem da unidade 2 vai ser "pegar o papel do token e comparar com esta matriz".

9. ESTRUTURA DO PROJETO E TECNOLOGIAS

9.1 Estrutura de pastas

Seguimos a estrutura sugerida em aula, separando responsabilidade por pasta. O desenho abaixo mostra o projeto completo, incluindo a pasta do Alembic e a pasta docs com esta documentação:

```

esuda-certificados/
├── alembic/
│   ├── versions/           # migrations (a inicial já está criada)
│   ├── env.py              # configuração do alembic (importa os models)
│   └── script.py.mako
├── alembic.ini              # configuração do alembic (url do banco fica aqui)
├── app/
│   ├── api/
│   │   └── v1/
│   │       ├── routers/    # as rotas, um arquivo por entidade
│   │       │   ├── usuarios.py
│   │       │   ├── eventos.py
│   │       │   ├── inscricoes.py
│   │       │   ├── certificados.py
│   │       │   └── categorias.py
│   │       └── api.py      # junta todos os routers em um só
│   ├── core/
│   │   ├── config.py       # configurações do projeto (lê o .env)
│   │   ├── database/
│   │   │   └── conexao.py  # engine, sessão e Base do SQLAlchemy
│   │   ├── models/        # tabelas do banco (SQLAlchemy)
│   │   ├── schemas/       # validação de entrada/saída (Pydantic)
│   │   └── main.py         # cria o app FastAPI e liga as rotas
├── docs/                   # toda a documentação da entrega
├── requirements.txt
└── README.md
  
```

Como funciona na prática: a requisição chega na rota (routers), os dados passam pela validação do schema (schemas), a rota aplica as regras de negócio consultando o banco pelos models (models) e devolve a resposta no formato do schema. Na segunda entrega a ideia é separar as regras de negócio em uma camada de services e os acessos ao banco em repositories, além de criar a pasta tests, como o professor comentou.

9.2 Tecnologias e justificativa

A stack foi escolhida dentro do que a disciplina pede (Python com FastAPI) e complementada com ferramentas que resolvem partes específicas do problema, como ORM, migration e validação. A tabela mostra o que cada tecnologia faz no projeto e por que a escolhemos.

Tabela 14 – Tecnologias do projeto e justificativa

Tecnologia	Uso no projeto	Por que escolhemos
Python 3	Linguagem do backend	Foi a stack definida na disciplina
FastAPI	Construção da API REST	Gera o Swagger automaticamente, validação integrada com Pydantic e curva de aprendizado boa
Uvicorn	Servidor de desenvolvimento	É o servidor padrão recomendado na documentação do FastAPI
SQLAlchemy	ORM (mapeamento objeto-relacional)	Trabalhamos com tabelas como classes Python, sem escrever SQL solto no meio das rotas
Alembic	Migrations	O banco evolui versionado; a migration inicial das 5 tabelas já está no repositório
Pydantic	Validação dos dados	Já vem dentro do FastAPI; valida tipo, campo obrigatório, e-mail e as regras RN05/RN06
SQLite	Banco na fase de desenvolvimento	Não precisa instalar nada para rodar e demonstrar (o arquivo esuda.db é criado sozinho)

PostgreSQL	Banco de produção (planejado)	É o banco relacional pedido para a versão final; para trocar é só mudar a DATABASE_URL no .env
JWT (python-jose/passlib)	Autenticação	Fica para a segunda entrega, como combinado
Pytest	Testes automatizados	Fica para a segunda entrega; por enquanto os testes foram feitos à mão no Swagger

10. ESTRATÉGIA DE BANCO DE DADOS E MIGRATIONS

O banco usado nesta fase é o **SQLite** (arquivo esuda.db), porque qualquer pessoa do grupo consegue rodar e demonstrar o projeto sem instalar servidor de banco. A configuração foi feita para aceitar qualquer banco por variável de ambiente: criando um arquivo .env na raiz com a linha DATABASE_URL=postgresql://usuario:senha@localhost/esuda, o mesmo código roda no PostgreSQL sem mudar mais nada. Isso resolve a migração para o banco de produção previsto para a versão final.

As tabelas iniciais são as cinco descritas nos capítulos 5 e 6: usuarios, categorias, eventos, inscricoes e certificados, com os relacionamentos do modelo. Sim, o projeto usa migrations. A migration inicial já foi criada e aplicada com o Alembic, usando os comandos abaixo:

```
# comando que usamos para gerar a migration inicial
alembic revision --autogenerate -m "cria tabelas iniciais"

# comando para aplicar
alembic upgrade head
```

O arquivo gerado está na pasta alembic/versions/ e está versionado no repositório, então quem clonar o projeto já encontra a estrutura completa do banco. De agora em diante, qualquer mudança de tabela entra como uma nova migration, nunca alterando o banco "no dedo". Isso garante que o banco de cada integrante do grupo evolua da mesma forma.

11. BACKLOG DO PROJETO

Backlog inicial com as histórias de usuário do projeto. A coluna "Entrega" marca em qual unidade planejamos fazer cada uma, e a prioridade ajuda a decidir a ordem dentro de cada entrega.

Tabela 15 - Backlog do projeto (histórias de usuário)

ID	História de usuário	Prioridade	Entrega
US01	Como participante, quero me cadastrar com nome, e-mail e senha para acessar o sistema	Alta	1
US02	Como admin, quero cadastrar usuários definindo o papel (admin/organizador/participante)	Alta	1
US03	Como admin, quero gerenciar categorias para organizar os tipos de evento	Média	1
US04	Como organizador, quero criar eventos acadêmicos com data, local e capacidade	Alta	1
US05	Como organizador, quero cancelar um evento para avisar que ele não vai mais acontecer	Média	1

US06	Como participante, quero listar os eventos para ver o que está acontecendo na instituição	Alta	1
US07	Como participante, quero me inscrever em um evento respeitando as vagas	Alta	1
US08	Como participante, quero cancelar a minha inscrição para liberar minha vaga	Alta	1
US09	Como organizador, quero ver a lista de inscritos do meu evento para preparar a entrada	Alta	1
US10	Como organizador, quero emitir certificado com código único para os participantes	Alta	1
US11	Como participante, quero consultar meu certificado pelo código	Média	2
US12	Como usuário, quero fazer login com JWT para o sistema saber quem eu sou	Alta	2
US13	Como sistema, quero bloquear as rotas conforme a matriz de permissões	Alta	2

US14	Como desenvolvedor, quero testes automatizados com Pytest nas regras de negócio	Média	2
------	---	-------	---

O corte entre entrega 1 e entrega 2 seguiu o que foi combinado em aula: na primeira fica tudo que depende só de CRUD e regras de negócio; login, autorização e testes automatizados ficam para a segunda unidade. Por isso as histórias US01 a US10 já estão implementadas nesta entrega, e as demais continuam no backlog com prioridade definida.

12. CRITÉRIOS DE ACEITAÇÃO

CrITÉrios de aceitação por funcionalidade. Usamos esses critérios como roteiro de teste no Swagger antes de fechar a entrega: para cada um, testamos o caminho certo e o caminho errado, conferindo o status devolvido.

12.1 Cadastro de usuário (US01)

Para o cadastro de usuário, os critérios que definimos são:

- Deve exigir nome, e-mail e senha;
- Não deve permitir e-mail duplicado (retorna 400);
- Não deve aceitar e-mail com formato inválido (retorna 422);
- A senha não pode aparecer em nenhuma resposta;
- Deve retornar 201 quando cadastrar certo.

12.2 Criação de evento (US04)

Para a criação de evento, os critérios são:

- Deve exigir título, data, local e capacidade;
- Capacidade deve ser maior que zero (senão 422);
- Não pode aceitar data de evento no passado (senão 422);
- Se mandar categoria_id que não existe, retorna 400;
- O evento nasce com status "ativo";

- Deve retornar 201 quando criar certo.

12.3 Inscrição em evento (US07)

A inscrição concentra várias regras de negócio, então é a funcionalidade com mais critérios:

- Só funciona com usuário e evento existentes (senão 404);
- Não pode inscrever o mesmo usuário duas vezes no evento (400);
- Não pode passar da capacidade do evento (400);
- Não pode inscrever em evento cancelado (400);
- Não pode inscrever em evento que já aconteceu (400);
- Deve retornar 201 com status "ativa" quando der certo.

12.4 Cancelamento de inscrição (US08)

Para o cancelamento de inscrição, definimos:

- Inscrição inexistente retorna 404;
- Cancelar muda o status para "cancelada" e mantém o registro no banco;
- Cancelar uma inscrição já cancelada retorna 400.

12.5 Emissão de certificado (US10)

Para a emissão de certificado, os critérios são:

- Só emite para inscrição existente (senão 404);
- Só emite para inscrição ativa (inscrição cancelada retorna 400);
- Não pode emitir certificado duplicado para a mesma inscrição (400);
- O certificado deve sair com código único e retornar 201.

12.6 Exclusões

Nas exclusões, o objetivo é não perder histórico, então:

- Evento com inscrição cadastrada não pode ser excluído (400);
- Usuário com inscrição cadastrada não pode ser excluído (400);
- Categoria com evento ligado não pode ser excluída (400);
- Exclusão feita com sucesso retorna 204.

12.7 Documentação (RF16)

Por fim, a documentação automática também tem critérios de aceitação:

- A rota /docs abre e mostra todas as rotas agrupadas por entidade;
- Dá para testar qualquer rota direto pelo Swagger sem instalar nada.

13. COMO RODAR O PROJETO

O projeto foi testado com Python 3.10+ e não depende de nenhum serviço externo. Para rodar na máquina local, os passos são:

1. **Criar o ambiente virtual:** `python -m venv venv`
2. **Ativar o ambiente:** no Linux/Mac, `source venv/bin/activate`; no Windows, `venv\Scripts\activate`
3. **Instalar as dependências:** `pip install -r requirements.txt`
4. **Criar as tabelas com a migration inicial:** `alembic upgrade head`
5. **Subir o servidor:** `uvicorn app.main:app --reload`

Pronto. Abrindo `http://localhost:8000/docs` no navegador aparece o Swagger, com todas as rotas prontas para testar sem instalar nada. Os critérios do capítulo 12 podem ser conferidos direto por lá.

Se quiser usar outro banco em vez do SQLite, basta criar um arquivo `.env` na raiz do projeto com a linha `DATABASE_URL=url_do_banco` (exemplo: `DATABASE_URL=postgresql://usuario:senha@localhost/esuda`). Não precisa mexer em mais nada no código.

14. CONSIDERAÇÕES FINAIS

Nesta primeira entrega o grupo entregou o backend completo do Esuda Certificados: cinco entidades modeladas com DER e dicionário de dados, uma migration inicial versionada com Alembic, 22 rotas REST organizadas por entidade e 15 regras de negócio validadas nas rotas e nos schemas. O Swagger gerado automaticamente pelo FastAPI ficou disponível em /docs e foi

usado como ferramenta de teste de todos os critérios de aceitação do capítulo 12.

Na comparação com o que foi planejado no backlog, as histórias US01 a US10 estão concluídas, e o restante (login com JWT, bloqueio de rotas pela matriz de permissões, consulta de certificado pelo código e testes com Pytest) segue para a segunda unidade, já com prioridade e ordem definidas. A matriz de permissões e o campo papel no banco foram deixados prontos de propósito, para que a autorização da unidade 2 encaixe sem precisar refazer o que já existe.

Como evolução do código, o próximo passo combinado com o professor é separar as regras de negócio em uma camada de services e os acessos ao banco em repositories, além de começar a suíte de testes automatizados. Com isso o projeto fica no formato esperado para a entrega final, com banco PostgreSQL configurado pela variável de ambiente.